



`struct` 内存分布: 每个变量需要地址开头为自身大小的倍数. 结构体整体的大小会是最大变量的倍数. 最优化: 变量从大到小排序.

Big Endian: 数位中不显著的 byte 放在大地址; Little Endian: 不显著的 byte 放在小地址. e.g. 0x1234 -> 0x12 | 0x34; 0x34 | 0x12.

RISC-V 架构: 32 reg, reg0 always 0. 指令后加 i 表示 rhs 为常数.

`add x y z`: `x = y + z`; , `sub`, `xor`, `sll`: shift left logically, `sra` shift right arithmetically.
`lw x offset(y)`: `x = *(y+offset)`; .
`sw x offset(y)`: `*(y+offset) = x`; .
`beq x y L`: jump to `L` if `x == y`. `j L`: jump to `L`.

内存分配: Code: 存程序; Static Data: 存全局变量; Heap: 存局部用指针分配的数组; Stack: 存函数内的变量; OS: 留给 OS.

Cache 结构: 将地址分为 Tag, Index, Offset. 将 Block 划分进不同的 Set.

Process: 资源分配单位; Thread: 执行单位.

`int fork()`: 将当前 process 复制一份. 返回 0 表示是子进程, 父进程返回子进程的 pid.
`int wait(int* status)`: 等待直到子进程之一执行完. 返回执行完的进程的 pid,
`int waitpid(int pid, int* status, int options)`.

child 在停止后仍然占用资源, 以让 parent 在之后知道运行结果. parent 停止后会杀死 child, 所有返回值由 pid=1 的进程接收.

RE: `[]` 表示其中之一, 内部不需要转义. `^` 表示排除.

`?` 表示出现 0/1 次, `+` 表示出现至少 1 次.

`|` 表示或.

`()` 为捕获组, 也表示优先级. `(?!)` 断言之后的不是内部的字符串, 且不消耗字符.

`tr '[:upper:]' '[:lower:]' < departments.txt | sort -u | awk '{print} END{print NR}'`: 将 txt 里的大写转小写; 排序并去重; 逐行输出并输出行数.

`lock`: `Acquire()` and `Release()`. `semaphore`: `P()` and `V()`. `condition`: `wait(&lock)`, `signal()`, `broadcast()`.

TCB: 先验地假设是安全的系统安全组件, 如 Kernel.

Virtual Memory: 以一个 page 作为单位, 一般是 4k; 对应 physical frame; 可以管理访问权限 (ACL), R/W/X. 利用这个技术可以延后大部分操作, 如内存的复制可以延后到写其中一个的时候; 分配新内存可以延后到写入内容的时候.

Page 的状态: Free, 未指向内存; Allocated, 指向内存; Swap out, 原本指向内存, 但由于长时间未访问, 指向的数据被放在 Disk 内称作 page file 的文件内, 下次访问的时候会产生一次 page fault 并从 Disk 内放回内存.

TLB: 缓存某一些虚拟地址对应的物理地址. 否则需要按字典树逐层查询.

Disk (机械硬盘): 形态呈多层轨道围绕一根柱子. 轨道叫做 platter, 有 2 个 surface, 每个 surface 有若干环叫做 track, track 分为若干个扇环叫做 sector, 不同 track 的同一个 sector 合称 cylinder, 若干个连在一起的 sector 合称 block, 是文件读写的最小单位.

一次读写: seek time, 旋转磁头到正确的 track; rotation: 等磁盘旋转到正确的 sector.

Disk (SSD): 4kB page 作为最小单元. 读: $20\mu s$, 写在空白 page: $200\mu s$, 写在非空白 page: 1.5ms (擦除). 故需要保留原数据, 新数据写在其他空白 page, 以 log 形式记录数据指针; SSD 中的 FTL 用来做这件事.

文件系统: 每个文件名对应一个 inode (对应关系存在 directory 内, 注意目录也是一个文件, 有对应的 inode), 内含指向文件内容地址 (同样以 block 为单位) 的指针 (12 个直接, 1 个一级间接, 1 个二级间接, 一层 1024 叉 (根据 block 大小算), 一个 block 4kB, 故 UNIX 内一个文件至多约 4GB), 以及权限信息和 hard link count 之类的.

复制文件的两种策略: Soft link, 新文件对应的 inode 存原文件的路径, 但没有原文件的内容; Hard link, 多个名字指向同一个 inode, 为了进行删除操作 inode 需要记录被多少个 hard link 指向.

指针的 new 需要有一段连续内存. 如果内存被以不同大小反复分配和释放会导致占用的部分在内存中很破碎. 故将不同大小的分配请求大致分类, 使得占用的部分相对规整.

Garbage collection: 自动分配内存, 不用手动 free, 如 python. 每过一段时间把无论如何都访问不到的内存 free 掉; 把没有指针指向的地址 free 掉 (环会倒闭, 故需要第一条).

IO Device: 如键鼠, 屏幕, 打印机等. 被 OS 视为物理内存来管理. 分类: Block/Stream: 是否能随机访问, e.g. Disk/Keyboard; Blocking/Non-Blocking/Asynchronous: IO 后要等待/不用等待, 一段时间没传完的数据截断/传完之后提醒.

Polling: 不断询问是否传输好, 适合传输速度固定可预测 (Disk). Interrupt: 适合不可预测

(Keyboard).

DMA: 帮助 CPU 访问 Memory 的 IO Device, 使 CPU 专注于计算. 使用虚拟内存访问 Memory, 否则 CPU 和 DMA 无法衔接.

Lan: 分两类. Hub, 用光纤串联各个设备, 即以太网, 如今被淘汰; Switch, 所有设备连接到 Switch, 使用 Mac 区分不同设备 (当某个设备只知道目标 IP 不知道 Mac 时用 ARP 广播询问), 即家中的网络交换机.

Wan: Router 作为传输节点, 使用 IP 区分设备.

TCP: 传输若干包之后返回是否全部接受, 接受则上调发送速率, 否则减半. e.g. SSH. UDP: 只提供端口供进程连接, 不保证丢包和速率. e.g. 直播推送.

网络七个层级: Application (HTTP), Presentation, Session, Transport (TCP), Network (IP), Datalink (Switch), Physical (Hub). 每一层 (除了 Physical) 都有对应的 Header 来标记数据去向和其他信息.

End-to-End Argument: 在 Application 层加密之后不需要在底层重复加密, 就算在底层加密后也需要在 Application 层加密 (不然 Header 是明文). 或者考虑传输失败的重试也同理, 但底层依旧可以重试, 因为知道具体重试哪个包, 可以提升效率.

Socket: 网络通信方式, 支持 RPC (remote process call), 即将远程函数调用包装成与本地函数调用相同, Stub 用来执行这个工作. Marshalling: 将函数参数变成可以在网络上传输的格式. Peer-to-Peer: 即 P2P, 网络通信架构.

Database 保证版本变化为 atomic: 也即随时断电依旧稳定, 不会有只修改了一部分文件的情况. 具体实现方式为: 先模拟操作, 将即将进行的修改记在 log 内, 最后记 log 结束; 再按照 log 从头执行, 再删去 log. 在修改单个文件的过程中也可能断电, 需要新建一个文件存放新内容, 再对新旧文件进行 rename. 这种策略称为 ACID.

Database 处理两个进程同时操作: 即有两段 R/W 序列. 对正确性的定义: 两段序列不交的两种情况 (一前一后) 都是对的. 希望两段序列能够相互穿插来减少 lock. 使用 2PL (2 phase lock): 一个 R/W 序列 release 一个锁之后不能 acquire 任意锁, 则是正确性的充分条件 (考虑不会同时有 A 先修改 B 再修改 和 B 先修改 A 再修改的情况, 修改-读取同理).

分布式系统文件管理: 两种方式. NFS: write 时将修改上传到 server, 定时向 server 询问文件是否修改, 读 server 的文件时指定文件和地址 (aka. cookies, 故 HTTP 也是 stateless 的), 使 server 不用记录任何信息. AFS: open 时缓存整个文件, close 时上传修改 (不保存文件修改的分支, 以最

后上传的为准) 并由 server 通知所有系统缓存失效, 下次 open 需要重新缓存; server 需要记录每个文件是谁上传的之类的信息, 但断电之后可以重建信息, 不需要 ACID. AFS 的策略被称作 BASE.